Web Services in 90 minutes

Focus on explaining a rule, tracing its effect and predicting the result. This guide covers the material actually present in the site's Weeks 1-6 review.

Set a 90-minute timer

0-10 min: this page • 10-25: HTTP • 25-40: REST and OpenAPI
40-55: PHP and middleware • 55-70: collections and cursors
70-80: formats and errors • 80-90: closed-book self-test

Processes and communication

A program is stored instructions. A process is a running instance with its own memory. Threads share their process's resources. Two Java processes do not share variables, even on the same laptop. They need explicit IPC: pipes, shared memory with synchronization, queues or sockets.

Client and server are roles: one requests, the other provides. A work queue passes jobs to workers; publish-subscribe sends events to independent subscribers; fan-out combines remote results. Distribution adds latency, concurrency and partial failures.

Sockets and TCP

An endpoint is an IP address plus a transport port. Many clients can connect to the same server port because their endpoint pairs differ. ServerSocket listens; accept() returns a separate connected Socket for one client.

```
Server: listen -> accept -> read -> process -> write
Client: connect -> write -> read -> close
```

TCP is an ordered byte stream. It does not preserve your write boundaries. Two writes can arrive in five reads. Reassemble complete messages using a delimiter or length prefix; UTF-8 defines the encoding, not the framing. A length prefix counts bytes.

```
CALC/1:  READY CALC/1 -> ADD 4 7 -> OK 11.0
         QUIT -> BYE
```

Flush buffered output after a complete message. readLine() == null means EOF; "" means an empty line. A sequential server finishes one session before handling the next. A worker pool adds concurrency but needs queue, timeout and shared-state controls. Blocking waits on a thread; asynchronous results arrive later.

The failure rule to remember

A timeout means the client stopped waiting. The server may already have committed the action. A retry is not automatically safe. UDP preserves datagram boundaries but supplies no delivery/order guarantee by itself.

Source: 511va.pages.dev/midterm/ | #start #systems #sockets

HTTP and state

Read the target and the headers

```
https://api.example.test:443/items?state=open#details
Request target: /items?state=open
Origin: https://api.example.test  (effective port 443)
```

The #fragment is not sent. A URI identifies; a URL locates; a URN uses a naming namespace. HTTP/2 uses TCP; HTTP/3 uses QUIC over UDP.

Request line: method, target, version. Response line: version, status. Headers, then a blank line, then optional body. Host identifies authority.

Accept requests response formats. Request Content-Type describes submitted content; response Content-Type describes returned bytes. Location identifies a created resource or redirect target.

Methods express intent

GET retrieves; HEAD retrieves metadata without response content; OPTIONS describes options. These are safe and idempotent. POST submits processing, often creation, and is not generally idempotent. PUT creates/replaces at a known target and is idempotent. PATCH applies partial changes and is not guaranteed idempotent. DELETE is idempotent.

Safe means retrieval intent, even if logs are written. Idempotent means the same intended effect after repetition. DELETE can return 204, then 404, and still leave the resource absent.

Statuses to distinguish

200 success or an empty collection • 201 created, with Location
202 accepted but unfinished • 204 success without content
304 reuse a validated cached representation, without content
400 malformed request/query • 422 parsed body has invalid shape/types
401 missing/invalid authentication, with challenge • 403 denied
404 missing target • 405 wrong method on known route, with Allow
406 no acceptable output • 415 unsupported input format
409 current-state conflict • 412 failed precondition
413 too large • 429 rate limited • 500 internal failure
502 bad upstream • 503 unavailable • 504 gateway timeout

301/308 are permanent redirects; 302/307 temporary; 303 points to a retrieval. Normal collection pages use 200, not 206, which is for range transfer. HEAD, 204 and 304 have no response content.

State caching and HTTPS

Transport, session and resource state differ. A session can outlive a connection. REST allows persisted resources; requests supply their needed context.

no-cache permits storage but requires validation before reuse. no-store forbids storage. private excludes shared caches. max-age sets freshness. ETag plus If-None-Match can yield 304. Strong ETags distinguish different bytes. Vary names selection headers; query variants are already in the request target.

Cookie flags: Secure restricts transmission to secure connections; HttpOnly blocks JavaScript access; SameSite affects cross-site sending. TLS checks the certificate chain, hostname and validity. HTTPS protects the channel, not ownership checks, SQL safety or seat allocation.

Source: 511va.pages.dev/midterm/ | #web #http #state

REST design and OpenAPI

Resource and representation

Section S102 is one resource. /v2/sections/S102 identifies it. JSON and XML are different representations of it. A resource can combine several tables. An enrollment is a relationship resource linking a student and section.

Use paths for identity/relationships and query parameters for collection views. Start design with actors, goals, domain resources and business rules. Functional requirements say what happens; non-functional requirements define qualities such as latency, privacy or consistency.

Know the REST constraints

Client-server separates concerns. Statelessness puts the needed context in each request. Cacheability states reuse rules. A uniform interface gives shared interaction semantics. Layers hide the full intermediary chain. Code on demand is optional.

The uniform interface has four parts: identify resources; manipulate them through representations; use self-descriptive messages; communicate available transitions through hypermedia.

Richardson levels: 0 = HTTP command tunnel; 1 = resources; 2 = HTTP methods/statuses; 3 = hypermedia. Level 3 does not prove every REST constraint. Noun-based URLs alone do not prove REST either.

OpenAPI describes the contract

```
openapi: 3.1.2
info: {title: Registration API, version: 1.0.0}
servers: [{url: http://127.0.0.1:8000/v1}]
paths:
  /sections/{id}:
    get:
      responses:
        "200": {description: Matching section}
        "404": {description: Missing section}
```

This abbreviated example omits input and response schemas. openapi names the format version; info.version names the description version; /v1 is a separate URL-version choice. Combine server URL plus path without duplicating /v1.

parameters covers path/query/header/cookie inputs; path parameters are required. requestBody covers content types and body schema. responses defines outcomes. components and $ref reuse definitions. The server must enforce all this; YAML and security declarations do not implement it.

Presence is separate from null

```
required: [id]
properties:
  id: {type: string}
  note: {type: [string, "null"]}
```

id must exist. note may be absent, a string or null. Add note to required to demand its presence. default/example annotations do not force server behavior. JSON "42" is a string, not integer 42.

Separate read, create and patch schemas; do not expose or accept every database field. Renamed fields, changed types, new required inputs and new enum values can break clients. Even added output fields can break strict closed-schema clients. A version label does not perform a migration.

Source: 511va.pages.dev/midterm/ | #rest #design #openapi

PHP persistence and middleware

Follow the responsibilities

Front controller boots the app. Router matches method and path. Controller validates and coordinates. Repository/PDO handles persistence. Mapper selects public fields. The response returns status, headers and content.

```
$stmt = $db->prepare(
  'SELECT id, capacity, seats_taken
   FROM sections WHERE id = :id'
);
$stmt->bindValue(':id', $id, PDO::PARAM_STR);
$stmt->execute();
$row = $stmt->fetch(PDO::FETCH_ASSOC);
```

A false fetch means no matching row. Otherwise map public fields, such as availableSeats = capacity - seats_taken. Bind values, not SQL structure. For column names or ASC/DESC, choose trusted expressions from an allowlist. PDO is not an ORM or authorization system.

Validate before writing

Check size and Content-Type, parse JSON, validate shape/types/required fields, then check references and business rules. Malformed JSON gets 400; valid JSON with the wrong property type gets the course's 422. Do not silently coerce "42" into 42.

Primary keys identify rows; foreign keys protect references; unique constraints block duplicates. A transaction groups related writes. Correct concurrency also needs a shared locking/isolation policy.

Last seat: A locks the section, rechecks capacity, inserts the enrollment, updates the counter and commits. B waits, then checks current state. Both seeing one seat earlier does not entitle both to enroll. Keep UNIQUE(student_id, section_id) even after input validation.

Failure before commit can roll back writes. Failure after commit cannot undo them. A lost reply or encoder error can leave a durable enrollment; retry may receive 409 rather than the original 201.

Predict middleware control flow

```
Success: A before -> B before -> handler -> B after -> A after
B returns 400 early: A before -> B before -> A after
Handler throws: ordinary after-blocks are skipped until a catch
```

finally still runs for entered layers during normal exception unwinding. An outer error layer may build a new 500 after an inner request-ID after-block was skipped. Put response finalization outside error conversion, or explicitly add the ID when building the error. Final-status logging must see the converted response.

PSR-7 withHeader()/withAttribute() return changed messages: capture the return value. PSR-15 defines middleware/handlers. Slim enters the last registered middleware first: add A, then B, then C means C enters first.

Parse before body validation; identity before identity-based rules. Return a response, not echo/exit; emit once at the boundary. Browser models do not execute PHP/DB/TCP. The local PHP example lacks complete authentication, ownership and prerequisite checks.

Source: 511va.pages.dev/midterm/ | #php #middleware

Collections pagination and cursors

The order of work matters

Apply trusted scope, then filters, total ordering, page slice and public representation. Filtering chooses rows; projection chooses properties. Sorting only after taking a page gives the wrong result.

```
GET /v2/sections?term=F26&state=open&minSeats=1
    &sort=-availableSeats,id&page=2&perPage=2
```

Filters combine with AND. The minus sign means descending seats; ascending id breaks ties. Matching ordered rows are:

```
S102 (4), S103 (4), S108 (4), S107 (2), S101 (1)
offset = (page - 1) * perPage = 2
Page 2: S108, S107   |   totalItems=5   |   totalPages=3
Status: 200          |   returned=2
```

Count before slicing. Navigation links preserve normalized filters, order and page size. A valid page beyond the last returns 200 with empty items and no next link. Numeric values outside the allowed bounds get 400.

The course query rules

Supported: term, state, minSeats, sort, page, perPage. Reject unknown, repeated, empty or array-shaped values with 400. Validate integers before casting; leading zeros are rejected. page: 1-10000. perPage: 1-100, default 20. minSeats: 0-500. These are this course's rules, not universal HTTP limits.

Offset can drift

```
First page: A, B | C, D
Insert X:   X, A | B, C | D
Page 2 now repeats B despite deterministic sorting.
```

A unique tie-breaker stabilizes unchanged data; it does not freeze insertions/deletions. Keyset pagination continues after the last returned ordering key instead of skipping a row count.

```
WHERE id > :lastId ORDER BY id ASC LIMIT :takePlusOne
```

For limit 2, fetching S103, S104, S106 means return S103/S104 and use S104 as the boundary. S106 is lookahead; using it as a strict-after boundary would skip it.

Composite order needs the whole tuple

For seats DESC, id ASC after (4,S103), select seats < 4 OR (seats = 4 AND id > S103). Group this OR when adding filters. Use the actual seat expression in WHERE; native PDO prepares need distinct placeholders for repeated values.

A cursor binds boundary/filter/order context. Validate it and recheck permissions. Base64 is encoding, not secrecy. A cursor is neither a credential nor a snapshot; mutable rows can still move. Consistent exports need a stable snapshot/versioned view.

Twenty returned rows can still cost a deep offset scan, an expensive sort/count or N+1 queries. Count and page must share scope/filters, but separate reads can see different states. Use suitable indexes, EXPLAIN and joins/batches without multiplying resources. Clients need page budgets, loop detection and safe link origins.

Source: 511va.pages.dev/midterm/ | #collections #cursors #journey

Formats errors and evidence

Negotiate the response without changing the data

Filtering and pagination choose the resources; serialization preserves their fields, values and order. Changing Content-Type alone does not convert JSON to XML. The course reads support JSON/XML/YAML; enrollment writes remain JSON-only.

```
XML sent to JSON-only write -> 415
Accept excludes every offered output -> 406
Accept: application/json;q=0.5, application/xml;q=1 -> XML
Accept: application/json;q=0, */*;q=1 -> XML, not JSON
```

For each offered type, the most specific matching range determines quality. q=0 excludes it, even with a broader wildcard. Highest eligible quality wins; ties prefer JSON, then XML, then YAML. Missing Accept defaults to JSON. Return Vary: Accept when selection depends on it.

Serialization traps

JSON: "4" differs from 4; [] differs from {}; absent differs from null. Sparse numeric PHP keys can turn a promised list into an object:

```
$json = json_encode(array_values($rows), JSON_THROW_ON_ERROR);
```

Use valid UTF-8. XML needs escaping and agreed types/list/null conventions; a namespace URI need not be downloadable. YAML needs careful quoting and restricted parsing. Bound bodies and disable unsafe external-entity or object-construction features.

Keep the client error useful and safe

Course mappings: malformed input 400; invalid parsed body 422; unsupported body 415; oversized content 413; duplicate/no seat 409; unexpected fault 500. Errors use application/problem+json even when a normal success would be XML/YAML.

```
{"type":"about:blank", "title":"Conflict", "status":409,
 "detail":"This section has no available seat.",
 "instance":"/requests/study-42"}
```

Problem Details has type, title, status, detail and instance. The actual HTTP status remains authoritative. Document extensions; clients should not parse free-text detail as a stable error code. Include a correlation ID, never credentials, SQL dumps or stack traces in the client body.

Logs are evidence with limits

Record selected request ID, method, route template, status, media type and duration. Bound and sanitize fields; avoid tokens, unrestricted bodies and unnecessary personal data. Protect log access and retention. A failed log sink must not mask the original fault. An operational log is not automatically an audit record.

PSR-3 levels, low to high: debug, info, notice, warning, error, critical, alert, emergency. Monolog implements logging features. A handler threshold controls what it records; severity is not the same as HTTP status.

Build the representation before emission. After partial output, a clean replacement error is difficult. A completion log does not prove delivery. A request ID correlates evidence; it proves neither identity nor ownership. Commit, encoding, logging and client receipt are separate events.

Source: 511va.pages.dev/midterm/ | #representations #errors #journey

Ten minute closed book check

Spend 6 minutes answering, then 4 checking. These are rewritten practice prompts, not the exam. The teacher's 36-question set is at https://511va.pages.dev/midterm/#practice.

Questions

1. Why can two TCP writes arrive through five reads? What supplies message boundaries?
2. Can a timeout prove a database write never happened?
3. Distinguish safe, idempotent, no-cache and no-store.
4. A route exists but rejects the method. Which status/header? How do 401/403 and 406/415 differ?
5. Is JSON plus noun-based URLs enough for REST? Name the four uniform-interface parts.
6. In OpenAPI, does nullable mean required? Does default insert a value?
7. Why bind SQL values and keep a unique constraint after validation?
8. A wraps B. B returns 400 early. Which after-code runs? What changes on an uncaught exception?
9. Give the course page-2 IDs, offset and totalPages. For limit 2 with three fetched rows, which is the cursor boundary?
10. What does a committed write followed by serializer failure imply? Does a completion log prove receipt?

Answers

1. TCP carries ordered bytes, not write boundaries. The application reconstructs frames with a delimiter or byte length. UTF-8 supplies encoding.
2. No. It only proves the local deadline expired; the write may have committed.
3. Safe: retrieval intent. Idempotent: repeated request has the same intended effect. no-cache: validate before reuse; storage allowed. no-store: do not store.
4. 405 with Allow. 401: missing/invalid authentication; 403: permission denied. 406: unacceptable output; 415: unsupported input.
5. No. Identify resources; manipulate through representations; self-descriptive messages; hypermedia-driven transitions. REST also constrains state, caching, layers and client/server separation.
6. No to both. required controls presence; a nullable type allows null. The implementation must apply defaults if promised.
7. Binding separates values from SQL structure; allowlist column/order choices. Database uniqueness protects against races and other write paths.
8. A's after-code runs on the returned 400; the handler is skipped. An exception skips ordinary after-blocks until a catch handles it, possibly losing an inner request-ID header.
9. S108 and S107; offset 2; three pages. Use the last returned row, never the extra lookahead row.
10. The commit remains durable. Before emission, a safe error may still be returned. A log does not prove the client received it.

One final request trace

Client intent -> HTTP target/headers -> middleware and route -> validation -> trusted query/transaction -> public mapping -> negotiated encoding -> middleware unwind -> emit. Explain what can fail at each point, and whether commit or emission has happened.

Source: 511va.pages.dev/midterm/ | #practice #reference | Review: https://511va.pages.dev/midterm/